

Introduction à Thymeleaf

1 QU'EST-CE QUE THYMELEAF ?

Thymeleaf est un moteur de templates Java moderne, qui s'exécute côté serveur, aussi bien dans des applications Web que dans des applications autonomes. Il sait produire du HTML, du XML, du JavaScript, du CSS et même du texte brut.

Sa particularité est la notion de Natural Templates : un template Thymeleaf reste un fichier HTML valide qui s'affiche correctement lorsqu'on l'ouvre directement dans un navigateur, sans serveur. Les designers peuvent donc travailler sur la maquette statique pendant que les développeurs y ajoutent la logique dynamique.

2 PLACE DE THYMELEAF DANS UNE ARCHITECTURE RÉPARTIE

Dans une application répartie, on distingue deux grandes approches pour produire l'interface :

Approche	Principe	Exemple
Rendu côté serveur (SSR)	Le serveur génère le HTML complet à partir d'un modèle de données	Spring MVC + Thymeleaf
Application monopage (SPA)	Le serveur expose une API REST/JSON, le client (React, Angular, Vue) construit l'interface	Spring Boot REST + Angular

Thymeleaf relève de la première approche. Il reste très pertinent pour les applications d'entreprise, les back-offices, les e-mails HTML et les interfaces enrichies avec des bibliothèques légères comme htmx, qui permettent de rafraîchir des fragments de page sans écrire de JavaScript complexe.

3 UN PREMIER TEMPLATE

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8" />
  <title>Liste des personnes</title>
  <link rel="stylesheet" href="../static/css/style.css"
        th:href="@{/css/style.css}" />
</head>
<body>
  <h1>Liste des personnes</h1>
  <a th:href="@{/addPerson}">Ajouter une personne</a>
  <table border="1">
    <tr>
      <th>Prénom</th>
      <th>Nom</th>
    </tr>
    <tr th:each="person : ${persons}">
      <td th:text="${person.firstName}">Prénom exemple</td>
      <td th:text="${person.lastName}">Nom exemple</td>
    </tr>
  </table>
</body>
</html>
```

Le moteur (Thymeleaf Engine) analyse le template, puis utilise les données Java (le modèle) pour remplacer les parties marquées par les attributs `th:*` et produire le document final.

Bonne pratique de sécurité. Utilisez `th:text` par défaut : il échappe le HTML et protège contre les failles XSS. Réservez `th:utext` (texte non échappé) aux contenus de confiance, jamais aux données saisies par un utilisateur.

4 INTÉGRATION AVEC SPRING BOOT

Dans un projet Spring Boot, il suffit d'ajouter le starter correspondant (Java 17 ou supérieur) :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

Les templates se placent par défaut dans `src/main/resources/templates/`, et les ressources statiques (CSS, JS, images) dans `src/main/resources/static/`. Aucune configuration supplémentaire n'est nécessaire.

Un contrôleur Spring MVC remplit le modèle puis retourne le nom du template :

```
@Controller
public class PersonController {

    private final PersonService service;

    public PersonController(PersonService service) {
        this.service = service;
    }

    @GetMapping("/persons")
    public String list(Model model) {
        model.addAttribute("persons", service.findAll());
        return "person-list"; // templates/person-list.html
    }
}
```

Point d'attention pour la migration : depuis Spring Boot 3 (Spring Framework 6), les API Servlet utilisent le paquet `jakarta.*` à la place de `javax.*`.

5 LES MODES DE TEMPLATE

Thymeleaf propose 6 modes de template (Template Modes) : HTML, XML, TEXT, JAVASCRIPT, CSS et RAW.

5.1 MODE HTML

Il accepte tout HTML : HTML5, HTML4, XHTML. Le moteur est tolérant : il ne valide pas le document et n'exige pas qu'il soit bien formé. Thymeleaf n'ajoute pas de nouvelles balises (à l'exception de `<th:block>`, conteneur invisible utile pour les boucles ou conditions sur plusieurs éléments) : il ajoute des attributs aux balises existantes.

Il en résulte le principe des Natural Templates :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>Mon premier template Thymeleaf</title>
</head>
<body>
  <h2>Bonjour Thymeleaf</h2>
  <!-- 'src' est utilisé quand on ouvre le fichier directement
        dans le navigateur -->
  <!-- 'th:src' remplace 'src' quand Thymeleaf traite le template -->
  
</body>
</html>
```

Pour une URL construite par l'application (ressource statique, route d'un contrôleur), on utilise la syntaxe `@{...}` : `th:src="@{/images/picture.png}"`. Pour une valeur lue dans le modèle, on utilise `${...}`.

5.2 MODE XML

Le document doit être bien formé : chaque balise ouverte est fermée, les valeurs d'attributs sont entre guillemets simples ou doubles. Dans le cas contraire, une exception est levée. En revanche, Thymeleaf ne valide pas le document par rapport à une DTD ou à un XML Schema.

```
<?xml version="1.0" encoding="UTF-8"?>
<persons>
  <person>
    <fname th:text="${personInfo.firstName}">Prénom</fname>
    <lname th:text="${personInfo.lastName}">Nom</lname>
    <country th:text="${personInfo.country}">Pays</country>
  </person>
</persons>
```

5.3 MODE TEXT

Il traite du texte brut (e-mails au format texte, par exemple) avec une syntaxe dédiée : `[[...]]` pour une sortie échappée, `[...]` pour une sortie non échappée, et `[# ...]` / `[/]` pour les structures de contrôle. Un fichier HTML ou XML traité dans ce mode est vu comme du simple texte : les balises sont ignorées.

```
Bonjour [(${customer.name})],

Voici la liste de nos produits :
[# th:each="p : ${products}"]
- [(${p.name})] : [(${#numbers.formatDecimal(p.price, 1, 2)})] EUR/kg
[/]

Cordialement,
La boutique Thymeleaf
```

5.4 MODE JAVASCRIPT

Il permet d'injecter le modèle dans des fichiers JavaScript. Le fichier est traité comme du texte, avec une syntaxe proche du mode TEXT. Les expressions sont placées dans des commentaires, ce qui garde le fichier valide en dehors de Thymeleaf :

```
// Template :
function showCode() {
    var code = /*[[${code}]]*/ '12345';
    document.getElementById('code').innerHTML = code;
}

// Résultat (code = 11223) :
function showCode() {
    var code = 11223;
    document.getElementById('code').innerHTML = code;
}
```

Exemple en ligne dans une page HTML :

```
<script th:inline="javascript">
    /*<![CDATA[*/
    var user = /*[[${someObject.userName}]]*/ 'Nom utilisateur';
    /*]]>*/
</script>

<!-- Résultat : -->
<script>
    var user = "Tom";
</script>
```

Thymeleaf sérialise correctement la valeur (guillemets, caractères spéciaux, objets convertis en JSON), ce qui évite les injections de script.

5.5 MODE CSS

Il fonctionne comme le mode JAVASCRIPT, pour des feuilles de style :

```
<style th:inline="css">
  .some-div {
    background-image: url([[@${someObject.imagePath}]]);
  }
</style>
```

5.6 MODE RAW

Aucun traitement n'est effectué. Ce mode sert à insérer telles quelles des ressources non contrôlées (fichier, réponse d'URL, contenu externe).

6 DIALECTES

Thymeleaf est extensible : un dialecte définit comment les templates sont traités (attributs, expressions, objets utilitaires). La bibliothèque centrale fournit le Standard Dialect, suffisant dans la majorité des cas, avec le préfixe th:.

Spring fournit le SpringStandard Dialect, qui est celui utilisé automatiquement avec Spring Boot. Il reprend le dialecte standard avec des adaptations propres à Spring, notamment :

- Les expressions sont évaluées avec SpEL (Spring Expression Language) à la place d'OGNL ;
- L'intégration avec les formulaires Spring (th:object, th:field, gestion des erreurs de validation) ;
- L'accès direct aux beans Spring avec la syntaxe `${@monBean.methode()}`.

Tout ce que vous apprenez avec le dialecte standard reste donc valable dans Spring.

Par ailleurs, des dialectes complémentaires existent, par exemple thymeleaf-extras-springsecurity6 (affichage conditionnel selon les rôles : `sec:authorize="hasRole('ADMIN')"`) ou des dialectes de mise en page.

7 LES SYNTAXES ESSENTIELLES

Syntaxe	Rôle	Exemple
<code>\${...}</code>	Expression de variable (modèle)	<code>\${person.firstName}</code>
<code>*{...}</code>	Expression sur l'objet sélectionné	<code>*{firstName}</code>
<code>@{...}</code>	URL	<code>@{/persons/{id}(id=\${p.id})}</code>
<code>#{...}</code>	Message internationalisé	<code>#{page.title}</code>
<code>~{...}</code>	Fragment	<code>~{fragments/header :: menu}</code>

Attributs fréquents : `th:text`, `th:each`, `th:if` / `th:unless`, `th:href`, `th:src`, `th:action`, `th:object`, `th:field`, `th:classappend`, `th:replace`, `th:insert`.

7.1 FRAGMENTS : ÉVITER LA DUPLICATION

```
<!-- templates/fragments/header.html -->
<nav th:fragment="menu">
  <a th:href="@{/}">Accueil</a>
  <a th:href="@{/persons}">Personnes</a>
</nav>

<!-- Utilisation dans une page -->
<div th:replace="~{fragments/header :: menu}"></div>
```

Les fragments sont aussi la base de l'approche htmx : un contrôleur peut retourner uniquement `"person-list :: rows"` pour mettre à jour une partie de la page.

7.2 FORMULAIRE LIÉ À UN OBJET

```
<form th:action="@{/persons}" th:object="${person}" method="post">
  <input type="text" th:field="*{firstName}" />
  <span th:if="${#fields.hasErrors('firstName')}"
        th:errors="*{firstName}">Erreur</span>
  <button type="submit">Enregistrer</button>
</form>
```

8 POINTS D'ATTENTION DE LA VERSION 3.1

- Pour des raisons de sécurité, Thymeleaf 3.1 restreint certaines expressions : l'accès direct aux objets `#request`, `#session` ou `#servletContext` n'est plus autorisé dans les templates. On transmet explicitement les données voulues via le modèle.
- Les expressions de type « appel de méthode statique » sur des classes arbitraires (`T(...)`) sont limitées.
- Avec Spring Boot 3 et plus, l'intégration Spring Security utilise `thymeleaf-extras-springsecurity6`.

9 RÉSUMÉ

- Thymeleaf est un moteur de templates côté serveur, fondé sur des *Natural Templates*.
- Il propose 6 modes : HTML, XML, TEXT, JAVASCRIPT, CSS, RAW.
- Le dialecte standard est étendu par SpringStandard Dialect (SpEL, formulaires, beans).
- `th:text` échappe le contenu ; `th:utext` est à utiliser avec prudence.
- Avec Spring Boot, la configuration se résume à l'ajout du starter.

10 EXERCICES PROPOSÉS

1. Créer un projet Spring Boot avec le starter Thymeleaf et afficher une liste de personnes dans un tableau.
2. Extraire l'en-tête et le pied de page dans des fragments réutilisés par deux pages.
3. Ajouter un formulaire de création avec validation (`@NotBlank`) et affichage des erreurs.
4. Générer un e-mail en mode TEXT avec la liste de produits d'une commande.
5. Comparer le même écran réalisé avec Thymeleaf (SSR) et avec une API REST consommée par un client JavaScript : discuter des avantages et inconvénients dans un contexte réparti.